

Ops & automation associate case study

Overview

The MSO team at Nous handles everything that happens to a member after they sign up: managing their switches, dealing with providers, talking to them when things go wrong, resolving the edge cases that don't fit a happy-path automation. As we scale, more of that work needs to move out of inboxes and into AI-assisted workflows.

This case study is the job in miniature. You take a real operational problem, work from a process doc, build the workflow in n8n (which is what we use), and make customer-facing judgement calls.

It's not a coding test. It's a thinking-and-judgement test. Please use AI tools heavily. That's how we work. We care about how you use them, not whether you avoid them.

We expect you to spend a few hours on this task. It's fine to leave things unfinished. We care more about your judgement than your coverage.

Setup

- **n8n**: free 14-day trial at n8n.io (no card needed). All build work happens here.
- **AI tools**: please use them. If you don't already have ChatGPT Plus or Claude Pro, sign up and screenshot the receipt and we'll reimburse you.
- **Mock APIs**: we've stood up a mock environment. Base URL, auth and endpoints below.
- **No real customer data**. Everything is synthetic.

The scenario

Nous helps members switch to better mobile (SIM-only) deals. We let users know we've started to look for a better deal for them to move to, and then attempt to

generate a recommendation automatically. When a recommendation can't be made automatically, a ticket goes into HubSpot and someone on the MSO team picks it up.

Manual flow: MSO open our internal tool, look at the available deals at the customer's postcode (coverage is already factored in), check what they're currently paying, and decides whether anything's worth proposing. If yes, they click "Generate recommendation", which sends a proposal to the customer and closes the ticket as Success. If no, they send a message and close it as Switch Failure.

There's a wrinkle: deal inventory (from our provider Stickee) refreshes through the day. So if a first check finds nothing, it's often worth waiting and retrying before giving up.

The team does this around 248 times a month, ~1.4 minutes per ticket. About 47% currently end in failure, usually because no deal beats the customer's current price by enough.

Most of this is mechanical. The Head of Ops wants to automate it. **Your task is to build that automation.**

Below: a process doc, team conventions, mock APIs, and test scenarios. Use them, fill in the gaps, push back where they're wrong.

What you're building

A single n8n workflow that takes a HubSpot ticket on a webhook and runs the MSO's job end-to-end.

Process doc

The kind of internal page someone on MSO would point you at on day one. Various people have added to it over time and it isn't perfectly current.

What this ticket is

`MOBILE_SWITCH_OFFER_RECOMMENDATION`. Generated when a member is due a mobile recommendation. Subjects look like `MOB | {customer} | Make recommendation | Mobile # {N}`. The `#N` matters: households often have more than one mobile service and the ticket is about a specific one.

Households with no mobile service come through as `Mobile #No service found`. Don't automate those.

What MSO actually does

They open the household in our internal tool, look at the deals available at the postcode (coverage is already computed by this point), check what the member's paying, and decide whether anything's worth proposing. If yes, they generate a recommendation. If no, they send a message and close the ticket.

The judgement is mostly whether the saving is big enough to be worth disturbing the customer. A few things to know:

- We only switch members to networks we have commercial relationships with. The deal feed gives you everything Stickee returns, so you'll need to filter. Approved set: Talkmobile, Giffgaff, iD Mobile (network IDs 9, 13, 18).
- iD Mobile is on the approved list but we treat it differently. From an ops Slack thread a few weeks back, paraphrased: *"can we please stop pushing people onto iD unless it's properly cheaper, their support is bad and we lose them by month two."*
- We exclude the user's current provider.

Failures

When closing a ticket as Switch Failure, the system reads two properties off it to decide what to do next. Both need to be set **before** the stage moves. (There's an old post-mortem about a customer getting the wrong comms because only one was written.)

Allowed values:

- `email_switch_failure_reason`: `Mob - can't beat deal`, `Customer ineligible/can't process switch`, `Customer unresponsive`, `Customer rejects switch`, `N/A`
- `reporting_switch_failure_reason`: `INELIGIBLE_COULD_NOT_BEAT_CURRENT_DEAL`, `INELIGIBLE_IN_CONTRACT`, `MOB_MANAGEMENT_SWITCHED_OFF`, `UNABLE_REQUEST_OUT_OF_SERVICE_PARAMS`, `OTHER`

Pick whichever fits.

Team conventions

These are the engineering bar in production. We aren't going to list them out for you. **Part of the test is whether you know what that bar looks like for an operational workflow.**

What we will say: our **workflows have to survive the real world**. Things go **wrong**, get **re-dispatched**, **time out**, **return weird payloads**, end up on the **wrong branch**. We expect yours to be **safe to inherit**, **easy to debug**, and **safe to re-run**. Up to you how you get there.

In some **circumstances** we send **messages to customers over WhatsApp** to **explain** what's happened - you should **write these** and **add them to your workflow**. Customer messages **go via** **POST /comms/whatsapp**.

API reference

Base URL: `https://nous-case-study-saffron.vercel.app`
Auth: Authorization: Bearer `cs-saffron-t57hs0dtxz-2026`

Calls to `/deals/{householdId}` need an extra header: `X-Ticket-Id: {ticketId}`. The mock uses it for scenario state.

Method	Path	Purpose
GET	<code>/hubspot/tickets/{ticketId}</code>	Read ticket properties
GET	<code>/context/{householdId}</code>	Household and mobile services
GET	<code>/current-payment/{mobileServiceId}</code>	<code>{ amountInGbpPounds }</code> or <code>null</code>
GET	<code>/deals/{householdId}</code>	SIM-only deals with coverage joined
POST	<code>/recommendation</code>	Generate deal card, send proposal
POST	<code>/comms/whatsapp</code>	Send WhatsApp message
PATCH	<code>/hubspot/tickets/{ticketId}</code>	Update ticket / move stage
POST	<code>/slack/notify</code>	Post a structured alert

Response shapes:

- `GET /deals` returns `{ "deals": [...] }` (envelope, not a bare array)
- `GET /hubspot/tickets/{ticketId}` returns `{ "id": "...", "properties": {...} }` (HubSpot-shaped)
- `PATCH /hubspot/tickets/{ticketId}` accepts either `{ "properties": {...} }` or a bare properties object

Responses are deterministic per `householdId` / `mobileServiceId`. State (attempt counters, PATCHed properties) persists per `ticketId`. To re-run cleanly, use a fresh `ticketId` or ask us to reset it.

Sample webhook payload:

```
{
  "ticketId": "TICKET-12345",
  "householdId": "HH-001",
  "userId": "USR-001",
  "subject": "MOB | Jane Smith | Make recommendation | Mobile #1",
  "ticketCategory": "Mobile",
  "pipelineId": "228462820",
  "stageId": "390658766",
  "testMode": true
}
```

HubSpot stage IDs you'll need:

- Success: `446512118`
- Switch Failure: `409734350`
- Manual: `440806104`
- Waiting: `5060559097`

Test scenarios

Your **workflow** needs to **handle** **TICKET-001** through **TICKET-011**, **paired with** **HH-001** through **HH-011**. Each is a different shape of data. Some are happy-path, some aren't. We won't tell you which is which: figuring that out is part of the exercise.

If your workflow handles all eleven sensibly, you're in good shape. What "sensibly" means is what you defend at the onsite.

Sandbox

TICKET-012 through **TICKET-024** also exist (paired with **HH-012** through **HH-024**). They aren't part of the evaluated 11, but each one's a different shape of data the official set doesn't cover, in case you want extra ground to test your workflow against. Use them however helps, or ignore them entirely.

What to submit

1. **Your workflow export** (three dots top right → Download)
2. **Screenshots** of the workflow executing. Pick whatever best shows what you built.
3. **A short written doc** (max 2 pages):

Part A — Reflection

- The two most important judgement calls you made and why
- One thing in the process doc you'd push back on, or a question you'd want to ask the team
- What you'd build into v2 once you'd seen the workflow run for two weeks
- Three metrics you'd watch in the first two weeks live, and what would cause you to pause or roll back

Part B — Reply handling

A customer has just received your "no deal" message and replied. For each reply: should the workflow handle it automatically, should AI handle it with safeguards, or should it go to a human MSO? Why? Then: how would you know two weeks in if your classifier was getting any of these wrong?

1. *"But I just saw a cheaper deal on iD Mobile direct for £8 a month — why didn't you find that?"*
2. *"Wait — have you already switched me? I didn't agree to anything."*
3. *"Can you check again in a month?"*

Also note (a sentence or two) **how you used AI tools** during the task. No right answer, just curious.